

RESEARCH DOCUMENTATION & WORKING LOG

Fraudulent Donation Link Detection Using AI

An interpretable machine-learning system for detecting fraudulent donation and charity URLs

Author: Syed (Research Intern)

Supervisor: Dr. Yasir

Programme: UNMC Internship

Repository: github.com/Syed-Margub/fraudulent-donation-link-detection-using-ai

Document type: Process & methodology log (companion to the research paper)

Version: v2.0 (expanded from full repository review)

Date: July 2026

About This Document

This is the working research log for the project. Its purpose is to record everything that happens during development — the design decisions, the dataset work, the feature implementations, the experiments, the mistakes, and the fixes — so that when the research paper is written, the methodology, results, and limitations can be assembled from a complete and honest record.

It is not the research paper itself. It is the source material the paper is built from. Each major section notes which part of a typical cybersecurity paper it feeds into.

This version (v2.0) was expanded after a full review of the project repository — every notebook, script, dataset, and analysis output — so it now documents the actual implemented code, the exact result figures, and the exploratory work in the "New Features Research Lab", not just the plan.

Document Changelog

Version	Date	Summary
v1.0	Jul 2026	Initial consolidation: problem framing, dataset strategy, feature list, the has_https finding, the data-corruption incident, and headline results.
v2.0	Jul 2026	Full-repository review. Added exact feature implementations, the testing methodology, precise result figures, three new non-linear feature findings (num_subdomains, tld_length, two_letter_tld), the "New Features Research Lab" experiment log, external CIC data integration, and a planned-vs-actual repository map.

Table of Contents

1. Introduction & Problem Statement

Feeds paper section: Introduction

Online giving is now a primary channel for humanitarian response — disaster relief, medical emergencies, and charity appeals are routinely funded through donation links shared on social media and messaging platforms. This trust is exploited by fraudsters who build fake donation pages that impersonate real charities or manufacture emotional urgency to extract money and payment details from well-meaning donors.

Existing phishing-detection systems are effective but domain-agnostic: they separate phishing from legitimate sites in general, with no notion of the donation context. They therefore miss signals specific to charity fraud — manipulative cause-urgency language, absent charity-registration details, mismatches between the domain registrant and the claimed cause, and suspicious payment pathways.

Problem statement. This project develops an interpretable machine-learning system that detects fraudulent donation links by combining an established phishing-detection baseline with donation-specific features that existing detectors do not model.

2. Research Objectives & Contribution

Feeds paper section: Introduction (Contributions)

2.1 Objectives

- Detect fraudulent donation links using a multi-layered detection framework.
- Extract structural, temporal, reputational, and textual features from donation URLs and their webpages.
- Train an interpretable Decision Tree that produces explainable decisions suited to a trust-and-safety setting.
- Evaluate performance with cross-validation and standard metrics, and analyse feature importance.

2.2 Core Contribution

The genuine novelty is four donation-specific features absent from prior phishing-detection literature. Standard lexical, URL-structure, and WHOIS/registrar features are treated as an established, cited baseline — not as contributions.

#	Novel feature	What it captures
1	Emotional-pressure / cause-urgency language	Manipulative, guilt-driven fundraising language designed to rush the donor.
2	Charity registration / tax-ID presence	Whether the page displays registration numbers or tax identifiers that real charities show and fakes omit.
3	Registrant-to-cause consistency	Whether the domain registrant matches the claimed charity (impersonation / look-alike check).
4	Payment-pathway type	Whether funds route via a vetted processor or via gift cards, crypto, wire, or personal accounts.

Honest scoping principle. Novelty is claimed only where it is real: the four features above are the contribution; everything else is baseline with citations. This is a deliberate stance maintained throughout the project.

3. Related Work, Reference Tools & Research Gap

Feeds paper section: Related Work & Background — to be expanded during the formal literature review

Machine-learning phishing detection is a mature field. Large public datasets such as PhiUSIIL (~134,850 legitimate and 100,945 phishing URLs) support high-accuracy classifiers using lexical, host-based, and content-based features. Registrar- and TLD-level reputation is also established, with abuse concentrated among specific registrars and TLDs in industry reporting.

3.1 Industry reference tools that shaped the design

The four-module architecture was informed by widely used cybersecurity tools; each maps to a module in this project:

Reference tool	Role	Maps to module
PhishTank, URLVoid	Community/aggregated blacklists of reported phishing URLs	Blacklist check
VirusTotal, ScamAdviser	Multi-engine URL reputation & trust scoring	URL structure / reputation
ICANN WHOIS lookup	Domain registration & age	Domain intelligence
CheckPhish, Google Safe Browsing	AI/content-based phishing analysis	On-page text (NLP)

3.2 The gap

This body of work is domain-agnostic. No public dataset or feature set targets donation fraud specifically, and no prior detector models the donation-specific signals in Section 2.2. This project sits precisely in that gap, adding the four novel features on top of a reproducible baseline. This section is a placeholder to be expanded into a full literature review with formal citations before submission.

4. System Design & Methodology

Feeds paper section: Methodology / Proposed Framework

A candidate URL is evaluated by four analysis modules whose outputs form one feature vector. An interpretable Decision Tree then maps that vector to a risk decision. The modular design mirrors the multi-view feature organisation common in the literature, extended with a reputation/blacklist view.

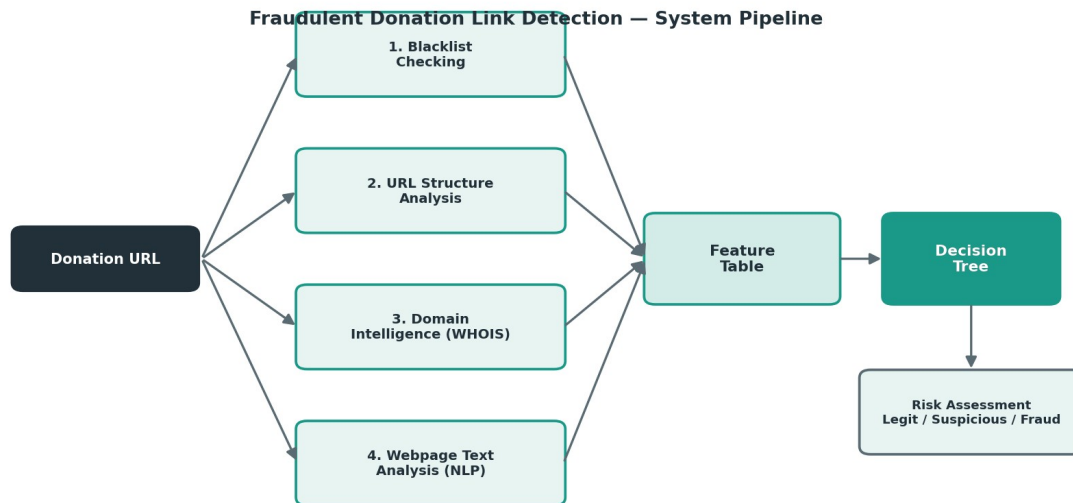


Figure 4.1 — End-to-end detection pipeline: URL → four modules → feature table → Decision Tree → risk assessment.

4.1 The four modules

- 1. Blacklist checking.** Reputation lookup against known phishing databases (PhishTank, OpenPhish, URLhaus). A direct hit is the strongest fraud signal; the limitation is that fresh scams are usually absent — which is exactly why ML is needed.
- 2. URL structure analysis.** Lexical features of the URL string, computed without visiting the page — so they work even for dead links.
- 3. Domain intelligence (WHOIS).** Temporal and reputational signals — domain age, expiry, registrar reputation, TLD risk.
- 4. Webpage text analysis (NLP).** On-page linguistic signals plus the four novel donation features. Under active development.

4.2 Classifier

A Decision Tree is used deliberately: it is interpretable (decisions trace through explicit rules), handles mixed numeric and binary features without scaling, trains quickly at this dataset size, and manages mild imbalance via balanced class weighting. Interpretability matters because a fraud verdict should be explainable to a human reviewer, and — as the results below show — because several features are informative in non-linear ways a tree can exploit but a linear model cannot.

5. Dataset Construction

Feeds paper section: Dataset Construction (a central methodology section)

Dataset work has been the most substantial and iterative part of the project. The decisions and their justifications are recorded in full because dataset quality — and the biases uncovered during curation — are themselves findings.

5.1 Strategy: phishing-vs-legitimate, not donation-only

A donation-only dataset does not exist publicly and would be too small to train reliably. The chosen strategy trains on a general phishing-vs-legitimate dataset, justified as follows:

1. Fraudulent donation links are a sub-category of phishing — same techniques (new domains, suspicious structure, urgency language, absence from trusted registries).
2. A detector that learns the phishing/legitimate boundary generalises to donation fraud.
3. Donation relevance is preserved by design: charity-themed URLs appear on both sides, so the model cannot learn a "charity-themed = safe" shortcut.
4. The four novel features then add explicit donation-domain intelligence on top of this baseline.

Status: agreed with the supervisor and documented in a separate justification report (Research_Contribution_Report) and email.

5.2 Sources and label convention

- **Fraudulent (label 1):** PhiUSIIL phishing entries and phishing feeds (a ~299-URL working phishing pool is kept in the repo).
- **Legitimate (label 0):** verified-legitimate PhiUSIIL entries plus hand-collected charity URLs (e.g. Asheville Humane, Ohlone Humane Society, Jacksonville Humane, ARL Boston) and major organisations (UNICEF, Red Cross, WHO, Save the Children, Doctors Without Borders).

Label convention: 0 = Legitimate, 1 = Fraudulent. PhiUSIIL uses the inverted convention (1 = legitimate), so labels were remapped. All features and reporting assume the remapped convention.

5.3 Liveness verification and balancing

Because the NLP module and the four novel features require the live webpage, every URL is passed through an automated liveness checker (`check_live_links.py`): it requests each URL concurrently and treats 404/410 responses and connection failures as dead, keeping everything else. Dead links are removed; the set is de-duplicated and balanced.

Current master list (`dataset_live_links_v2.csv`):

Metric	Value
Total URLs	804
Fraudulent (label 1)	441
Legitimate (label 0)	363
Class balance	1.21 : 1
URLs served over <code>http://</code> vs <code>https://</code>	280 / 524
Liveness	100% (dead links removed)

Note. The 804-row master list is augmented with ~60 verified legitimate http links (Section 5.4), giving the 863-row feature table used for the current results.

5.4 Finding: the has_https bias — a leakage problem and its fix

Observation. In the initial dataset, 100% of URLs without HTTPS were fraudulent. A feature that is perfectly predictive in training is almost always a warning sign of sampling bias, not a genuine law.

Diagnosis. The perfect split was a sourcing artefact: all legitimate URLs came from large, well-maintained charities that use HTTPS, while phishing feeds captured many links in raw http form. In reality most phishing sites now use HTTPS too (free certificates are standard), so the feature was unrealistic in both directions and the tree could have learned "no HTTPS = fraud" as a shortcut.

Fix. Legitimate http-serving examples were added. Candidates were extracted from a published phishing-detection dataset's legitimate class (~3,172 http candidates in `legitimate_http_sites_dataset_v2.csv`) and then verified live: each was requested with redirects followed, and kept only if it responded and did not redirect to https. ~60 survivors (`legit_http_sites_LIVE_v3.csv`) were added with label 0.

Result. The no-HTTPS scam rate fell from 100% to 82.6% (339 no-HTTPS URLs: 280 scam / 59 legit), and `has_https` moved from an effectively deterministic giveaway to a strong-but-non-deterministic signal. This is the intended outcome — the feature still carries information but no longer leaks the label.

Limitation. The legitimate http examples skew toward older, smaller, niche sites, so `has_https` remains somewhat elevated and mildly correlated with other features. A perfectly representative http sample is not achievable given how rare http-only legitimate sites now are; recorded as a known limitation.

5.5 Negative result: the Kaggle set cannot supply fraudulent donation links

A candidate phishing dataset (Kaggle "web-page phishing detection", stored as `http_dataset.csv`) was evaluated as a source of fraudulent donation links and rejected for two reasons:

- **No genuine donation-fraud content.** Of 5,715 phishing URLs, only 34 matched any donation/charity keyword, and nearly all were substring false positives or bank-credential phishing on compromised sites — not fake charity appeals.
- **Staleness.** The data is years old; phishing URLs are short-lived, so the links are dead — fatal for the on-page NLP and the four novel features that need a live page.

Conclusion: it is usable for legitimate examples (a legitimate site from a few years ago is usually still legitimate) but not for fraudulent donation links, which must be freshly collected while live. This directly motivates the current data-collection task.

6. Feature Engineering — Implementation

Feeds paper section: Methodology (Feature set & extraction)

Features are built in the main notebook (Feature Table.ipynb). This section documents the actual implementations and the working method, not just the feature names.

6.1 Working method: write → test on one URL → apply to all

Every feature followed the same disciplined pattern: define a function, test it on a single URL with print statements to confirm the output, then apply it across the whole table with pandas `.apply()`. This catches logic errors on one example before they propagate to 800+ rows.

```
def get_url_length(url):      return len(url)
def get_has_https(url):      return 1 if urlparse(url).scheme=="https" else 0
def get_num_subdomains(url):  return urlparse(url).netloc.count(".") - 1

# test on one URL first ...
df["url_length"] = df["url"].apply(get_url_length) # ... then apply to all
```

6.2 URL-structure features

- **url_length, has_https, num_subdomains** — parsed with `urllib.parse`.
- **special_character_count** — counts every non-alphanumeric character via regex.
- **suspicious_keyword_count** — counts matches of a scam lexicon in the URL: donate, urgent, help, verify, secure, account, login, gift, fund, claim.

6.3 Domain intelligence (WHOIS) — one lookup, three features

WHOIS lookups are the slow step (~25 minutes for the full set), so a single lookup returns age, expiry, and registrar together rather than three separate calls. A helper normalises WHOIS dates, which arrive inconsistently as lists, strings, or timezone-aware datetimes:

```
def _clean_date(dt):
    if isinstance(dt, list): dt = dt[0]
    if not isinstance(dt, datetime): return None
    if dt.tzinfo is not None: dt = dt.replace(tzinfo=None) # avoid tz mismatch
    return dt

# age, expiry, registrar all derived from ONE whois.whois(url) call
```

- **unknown_registrar** — 1 when WHOIS returns no registrar (e.g. free-hosting subdomains with no registration record of their own).

6.4 Reputation features from external data (Cybercrime Information Center)

Two features integrate published industry statistics — the Cybercrime Information Center registrar and TLD phishing tables (Feb–Apr 2026):

- **registrar_phishing_score** — the registrar's published phishing score. Matching WHOIS registrar names to the published names required a normaliser (`clean_registrar`) that lowercases, strips bracketed tags and parentheses, removes corporate suffixes, and collapses whitespace. This was an improvised fix after raw names failed to match.

- **tld_phishing_score** — the TLD's published phishing score, after extracting the TLD from the URL with a regex.

Sentinel encoding. Missing data is encoded explicitly so it is never confused with a real score: registrar_phishing_score uses -2 (no WHOIS data) and -1 (registrar not in the tables); tld_phishing_score and the WHOIS day-counts use -1. These sentinels are the source of the non-linear patterns in Section 7.

6.5 On-page text (in progress)

A concurrent scraper (get_page_text, BeautifulSoup, 20 threads) fetches visible page text; this feeds the planned text features (urgency_word_count, emotional_word_count, call_to_action_count, text_risk_score) and then the four novel donation features. Raw page text is deliberately kept out of the saved feature table (see Section 9).

6.6 Feature inventory

Module	Feature	Status
URL	url_length, has_https, num_subdomains, special_character_count, suspicious_keyword_count	Implemented
Domain	domain_age_days, domain_expiry_days, unknown_registrar, registrar_phishing_score, tld_phishing_score	Implemented
Experimental	tld_length, two_letter_tld	Implemented (analysis only)
Reachability	page_reachable	Implemented
Text (NLP)	urgency_word_count, emotional_word_count, call_to_action_count, text_risk_score	Planned
Novel	emotional-pressure, charity-registration, registrant-cause mismatch, payment-pathway	Planned (core contribution)

7. Feature Analysis & Results

Feeds paper section: Results & Discussion (Feature-importance analysis)

Results are from the current dataset snapshot after the has_https de-biasing. "Separating power" is the absolute correlation of each feature with the fraud label. Correlation captures only linear separation, so it must be read alongside the per-feature scam-rate plots — several features are informative in non-linear ways that correlation scores near zero but a Decision Tree can exploit.

7.1 Feature ranking (exact figures)

Feature	Correlation w/ scam	strength
domain_age_days	-0.529	0.529
has_https	-0.507	0.507
unknown_registrar	+0.212	0.212
domain_expiry_days	-0.180	0.180
tld_phishing_score	+0.170	0.170
url_length	+0.128	0.128
tld_length	+0.120	0.120
registrar_phishing_score	+0.097	0.097
special_character_count	+0.068	0.068
suspicious_keyword_count	+0.047	0.047
num_subdomains	-0.002	0.002
two_letter_tld	-0.001	0.001

Reading the signs. Negative correlations (domain age, has_https, expiry) mean higher values indicate legitimacy; positive ones (reputation scores, url length) mean higher values indicate fraud.

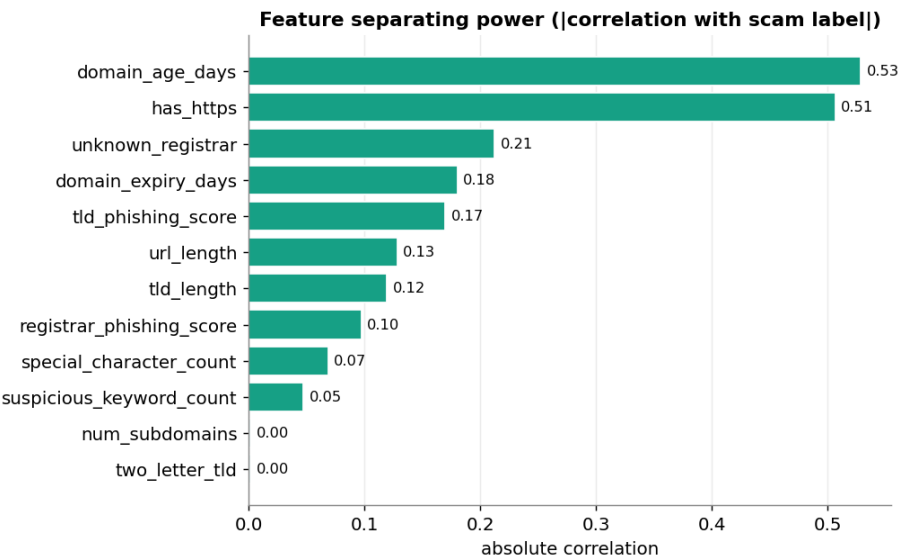


Figure 7.1 — Absolute correlation of each feature with the fraud label.

Readings: domain_age_days (0.53) is the strongest feature — a robust, hard-to-fake signal, since aged domains cannot be bought cheaply; a defensible position for the paper. has_https (0.51) remains strong but is no longer deterministic after de-biasing. unknown_registrar (0.21) and tld_phishing_score (0.17) carry real weight, validating the registrar/TLD work.

7.2 has_https before/after de-biasing



Figure 7.2 — has_https vs fraud. No-HTTPS: 82.6% scam (280/339); HTTPS: 30.7% scam (161/524). Before augmentation the no-HTTPS group was 100% fraudulent.

7.3 domain_age_days

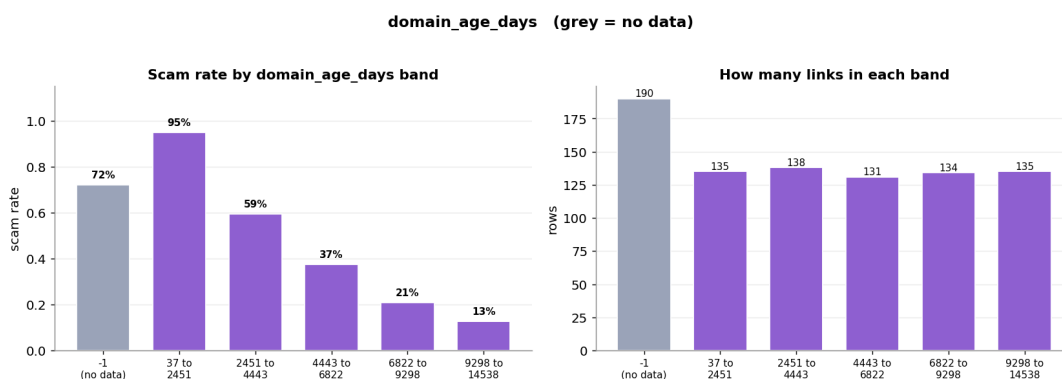


Figure 7.3 — Domain age vs fraud. Fraud clusters at very young ages; the -1 (failed WHOIS) bucket is itself 72% scam and shown separately.

7.4 New finding: num_subdomains is strongly non-linear

num_subdomains has a near-zero correlation (-0.002) and would be discarded by any linear ranking. But its scam-rate by value is strongly non-linear:

Subdomains	URLs	Scam rate
0	49	98.0%
1	662	45.3%
2	133	57.1%
3	19	89.5%

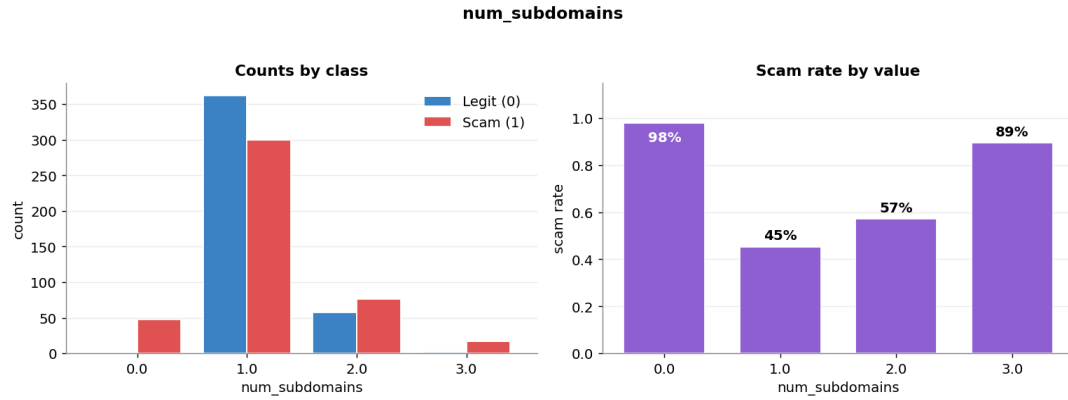


Figure 7.4 — num_subdomains. Both extremes (0 and 3+) are overwhelmingly fraudulent; the correlation of ~0 hides this completely.

Implication. This feature must be kept. It is a textbook case for using a Decision Tree (which can split on both extremes) and for reporting tree-based importance rather than correlation in the final paper.

7.5 New finding: long TLDs are almost always fraudulent

tld_length shows that unusual long TLDs are near-deterministic fraud markers, while common lengths are neutral:

TLD length	URLs	Scam rate	Example
2 (.uk, .ro)	237	48.9%	neutral
3 (.com)	593	49.4%	neutral
4 (.host, .site)	17	100%	fraud
5 (.online)	6	100%	fraud
6+	10	80-100%	fraud

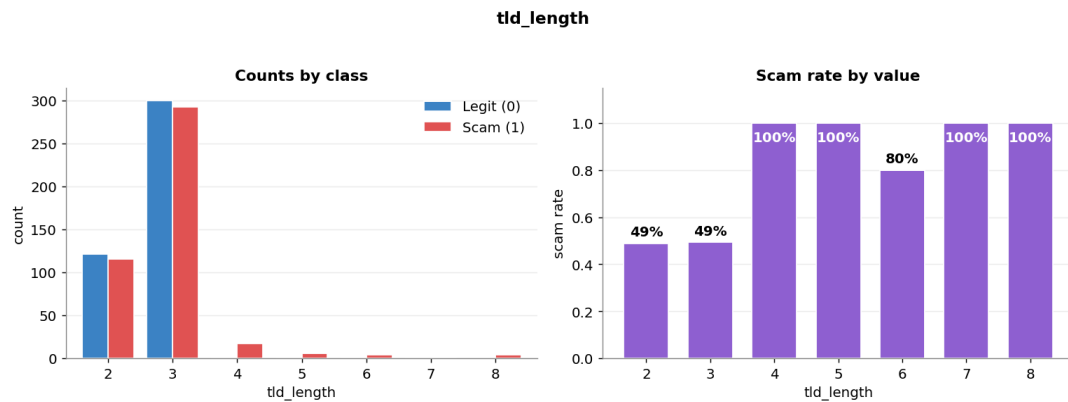


Figure 7.5 — tld_length. TLDs of length ≥4 are essentially all fraudulent, though they are rare.

7.6 registrar_phishing_score (non-linear / sentinel effect)

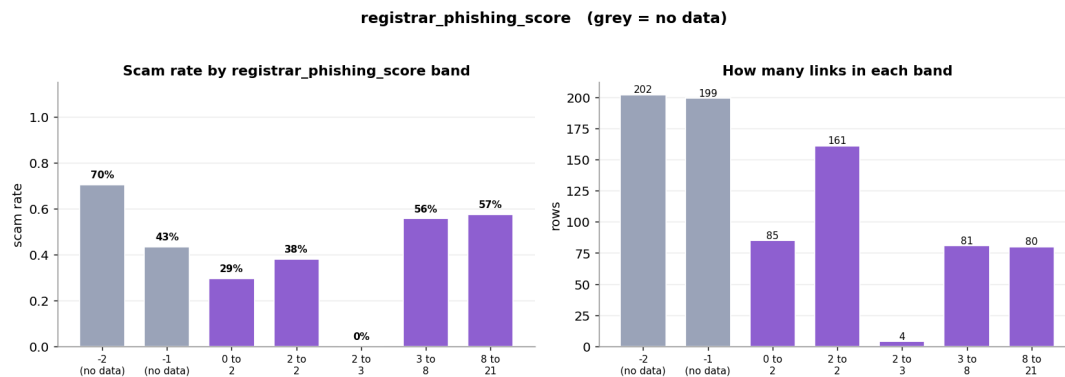


Figure 7.6 — Registrar phishing score. The "no data" sentinel (-2) is 70% scam; the score dips then rises for real values — a U-shape correlation reports as ~0.10 but a tree can split on.

7.7 Feature to retire: two_letter_tld

The experimental two_letter_tld flag shows no separation at all — 51.1% scam when 0 and 51.0% when 1. It is superseded by tld_length and tld_phishing_score and can be retired. It is retained in the log for transparency and because it validated the regex approach used elsewhere.

Overall methodological takeaway. Three of the twelve features (num_subdomains, tld_length, registrar_phishing_score) are informative only non-linearly. This is the central argument for the interpretable-tree choice and for reporting mutual-information / tree importance, not correlation, as the feature-importance measure in the paper.

8. Experimentation Log — "New Features Research Lab"

Feeds paper section: Methodology (feature exploration) / internal record

A dedicated notebook and folder (New Features Research Lab) is used to prototype and stress-test feature ideas before they enter the main pipeline. These exploratory studies are recorded here because they document the reasoning behind which features were kept, refined, or dropped.

Experiment	What was done	Outcome
Two-letter TLD filter	Regex-extracted all URLs whose host ends in a 2-letter TLD (245 URLs) and counted the legit/scam split.	No separation (~51% each). Idea generalised into <code>tld_length</code> ; flag retired.
HTTPS-scam investigation	Isolated the 161 HTTPS URLs that are scams (<code>https_scamURLs.csv</code>) from the 524-URL HTTPS subset (<code>Has_https_Dataset.csv</code>).	Confirmed empirically that scammers do use HTTPS, motivating the <code>has_https</code> de-biasing rather than trusting the raw signal.
HTML-ending URLs	Collected the 33 URLs ending in <code>.html</code> (<code>URLTLD_html.csv</code>) to check whether a page-extension cue carries signal.	Exploratory; noted for possible future lexical feature.
TLD length	Extended the 2-letter idea to full TLD length, surfacing the " <code>≥4 = fraud</code> " pattern now in Section 7.5.	Kept as an analysis feature.

9. Engineering Log — Challenges, Mistakes & Resolutions

Feeds paper section: Limitations / methodology honesty (and internal reproducibility)

Per the supervisor's instruction, mistakes and their fixes are documented as first-class content — the raw material for an honest methodology and limitations section.

Issue	Symptom	Cause	Resolution
WHOIS date/timezone handling	<code>domain_age_days</code> could silently return -1.	WHOIS dates arrive as lists, strings, or tz-aware datetimes; naive subtraction failed.	<code>_clean_date()</code> normalises all cases and strips tzinfo before subtracting.
Registrar name mismatch	Registrar scores failed to join to the CIC table.	WHOIS names carry bracketed tags, parentheses, and corporate suffixes absent from the published names.	<code>clean_registrar()</code> normaliser; improvised after inspecting the mismatches.
Regex character-class typo	Special-character counting was slightly off.	Pattern used a lowercase <code>z</code> in a range, creating an unintended ASCII span.	Corrected the character class; verified counts on known URLs.
<code>.str.extract</code> on a single string	<code>AttributeError</code> in the augmentation loop when extracting TLDs.	<code>.str</code> is a Series accessor; it was called on one URL string, not a column.	Use <code>re.search</code> on the single URL, matching <code>.str.extract</code> first-match behaviour.
page_text corrupted the CSV	Feature table exploded to 122 columns with 100+ empty columns; page text spilled into	Raw scraped text (commas, quotes, multilingual) was saved into the CSV, breaking row	Never store raw <code>page_text</code> in the feature CSV; save with <code>index=False</code> and

Issue	Symptom	Cause	Resolution
	wrong rows; non-UTF-8 bytes.	structure; Excel re-saved it as Windows-1252 (mojibake).	encoding="utf-8-sig"; do not re-save via Excel. A cleaned 863-row table was recovered.
Accumulating "Unnamed" columns	Extra index columns across saved versions.	Progressive saves (Feature Table → 4 → 5 → 6) used to_csv() without index=False.	Drop ^Unnamed columns and always save with index=False.

Root-cause pattern. Recurring themes: silent fallbacks (returning -1 on error, which can hide failures) and mixing free-form text with structured storage. Both are now guarded — errors are surfaced, and raw text is kept out of the feature table.

10. Tools, Environment & Reproducibility

Feeds paper section: Experimental Setup / Reproducibility

Category	Details
OS / IDE	Windows; VS Code with Jupyter notebooks; Anaconda.
Language	Python 3.
Libraries	pandas, urllib.parse, re, requests, BeautifulSoup (bs4), python-whois, scikit-learn, matplotlib, concurrent.futures.
External data	PhiUSIIL dataset; Cybercrime Information Center registrar & TLD phishing statistics (Feb–Apr 2026); Kaggle web-page phishing dataset (legitimate examples only).
Version control	Public GitHub repository with issue tracking for the sprint plan.
Key data files	dataset_live_links_v2.csv (804-row master); Feature Table*.csv (staged feature tables); legit_http_sites_LIVE_v3.csv (verified http additions).

Reproducibility note. WHOIS features are re-computable but slow (≈25 min for the full set), so the augmentation workflow extracts features for new rows only and appends them with identical logic to avoid a train/serve seam. WHOIS calls are wrapped in try/except and default to sentinel values on failure.

11. Repository Structure — Planned vs Actual

Feeds paper section: (internal tracking)

The original plan (Project Structure) is an aspirational layout with src/ modules and a tests/ suite. The actual repo is currently notebook-driven, with analysis outputs materialised to disk. Documenting the gap keeps the plan honest and flags the refactor still to come.

Planned (Project Structure)	Actual (current repo)
src/ modules: blacklist_check.py, url_analysis.py, domain_age.py, text_analysis.py, feature_pipeline.py, scoring.py, app.py	Feature logic lives in notebooks/Feature Table.ipynb; one standalone script (check_live_links.py).
tests/ unit-test suite	No formal unit tests yet; functions are validated by single-URL print tests inside the notebook.

Planned (Project Structure)	Actual (current repo)
notebooks/ (3 numbered notebooks)	notebooks/ with Feature Table.ipynb, Dataset Extraction.ipynb, feature_visualization.ipynb, and the New Features Research Lab folder.
results/figures + reports	outputs/feature_analysis/ with per-feature plots, summaries, and split CSVs.

Planned refactor. Once the feature set is final, the notebook logic should be extracted into the src/ modules and covered by the tests/ suite for a clean, reproducible pipeline.

12. Current Status

Stage	Status
Project setup & planning	Complete
Dataset collection, cleaning, balancing, liveness	Complete
URL-structure features	Complete
Domain / WHOIS / registrar / TLD features	Complete
External CIC reputation integration	Complete
has_https de-biasing (http augmentation)	Complete
Feature analysis & visualisation	Complete
On-page text (NLP) features	In progress
Four novel donation-specific features	Planned
Model training & evaluation	Planned
Literature review & gap analysis write-up	Planned
src/ refactor + unit tests	Planned

13. Next Steps

Feeds paper section: Future Work

1. Collect fresh, live fraudulent donation links from current feeds (PhishTank, OpenPhish, URLhaus) filtered for donation/charity keywords and verified live — the current blocker for the novel-feature case study.
2. Build the on-page text (NLP) features, then the four novel donation-specific features.
3. Train and evaluate the Decision Tree with K-fold cross-validation; report accuracy, precision, recall, F1, confusion matrix, and tree-based feature importance (not correlation).
4. Write the full literature review and gap analysis with formal citations.
5. Refactor notebook logic into src/ modules with a tests/ suite.
6. Assemble the manuscript from this log for one of the candidate venues.

14. References & Sources to Cite

Feeds paper section: References — verify full bibliographic details before submission

- Prasad, A. & Chandra, S. — PhiUSIIL: A diverse security profile empowered phishing URL detection framework based on similarity index and incremental learning. Computers & Security (2024). [PhiUSIIL dataset & framework]
- Interisle Consulting / Cybercrime Information Center — Phishing Landscape; registrar and TLD phishing statistics (Feb–Apr 2026). [registrar_phishing_score, tld_phishing_score]
- Prior work on lexical and registrar-based phishing features (e.g. PhishAri-style registrar signals; ICANN INFERMAL registration-behaviour analysis). [baseline features]
- Kaggle web-page phishing detection dataset (shashwatwork) — evaluated; used for legitimate examples only (Section 5.5).
- Reference tools that informed the module design: PhishTank, URLVoid, VirusTotal, ScamAdviser, ICANN WHOIS, CheckPhish, Google Safe Browsing.

Appendix A — Documentation-to-Paper Section Map

This document	Target paper section
1. Introduction	Introduction
2. Objectives & Contribution	Introduction (Contributions)
3. Related Work & Reference Tools	Related Work & Background
4. System Design & Methodology	Methodology / Framework
5. Dataset Construction	Dataset Construction
6. Feature Engineering (Implementation)	Methodology (Feature set)
7. Feature Analysis & Results	Results & Discussion
8. Research Lab experiments	Methodology (feature exploration)
9. Engineering Log	Limitations + methodology honesty
10. Tools & Environment	Experimental Setup / Reproducibility
13. Next Steps	Future Work
14. References	References

Appendix B — Repository File Inventory

Key files reviewed for this documentation:

Path	Purpose
README.md, DETECTION_APPROACH.md, DATASET_BUILDING_STAGE.md	Design & planning documents.
Reference Sites.md	Industry reference tools informing the module design.
data/dataset_live_links_v2.csv	804-row master URL+label list.
data/Phishing-RegistrarStats / TLDstats (CIC)	External reputation tables feeding the score features.
data/http_dataset.csv	Kaggle phishing dataset (legitimate-http source; Section 5.5).
check_live_links.py	Concurrent liveness verification script.
notebooks/Feature Table.ipynb	Main feature-engineering notebook (all functions).
notebooks/Dataset Extraction.ipynb	Extracts legitimate http candidates.
notebooks/New Features Research Lab/	Feature prototyping & experiments (Section 8).
notebooks/feature_visualization.ipynb	Generates the per-feature analysis plots.
outputs/feature_analysis/	Per-feature plots, summaries, and split CSVs.